

Project: Frog Story Gesture Capture Gloves

Date: 04/27/2026

Quickstart Guide

Initial Author: Erik Moore

Contact: * Phone ~ +1 (608) 780-9278 * Email ~ h65w675@student.montana.edu

Introduction

This guide is for whomever has the.. let's call it an opportunity to pick up the project following my semester of progress. It will firstly cover basic operation, followed by details of the current development avenues. Finally, I will cover what I believe are the most advisable next steps.

Startup

To begin, you should use the right-handed glove. This is the least problematic of the two options (given the current design), and slightly ahead in development (developing the left-hand glove beyond its current state was not possible due to bad hardware). Before plugging the glove in, ensure you have both Pycharm installed (or your python IDE of choice, but I used Pycharm, so you may run into small inconveniences with this guide if you opt for something else), as well as the Arduino IDE (this is non-negotiable).

Current Tasks ~ Install Pycharm and Arduino IDE:

1. Link to install Pycharm: [Jetbrains Installation](#)
2. Link to install Arduino IDE: [Arduino Support](#)

You should now have both of these tools installed on your system. Next, we will determine which COM port our (right-hand) glove connects to. To do this, we have two options. I will lay them out going forward; but first, as a developer you need to make a few quick installations in your arduino IDE. You should have already put the entirety of this project's code somewhere in your file system (personally, I created a folder off of the C:/ drive). If not, ensure that you download everything from the current implementation of the project on the microsoft teams (Nadya should have made you a member for you to have access to this). Additionally, I

have a slightly less organized version of everything on a [github](#) instance I created. However, I do recommend creating your own and starting from there.

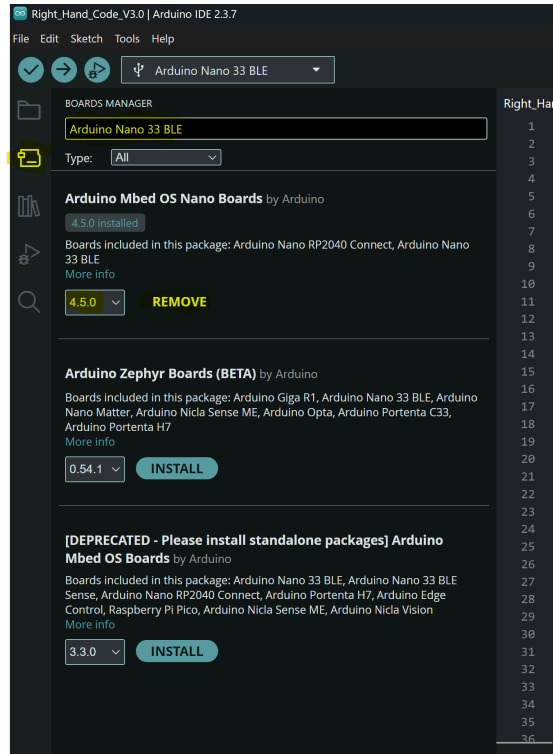
Current Tasks ~ Ensure all Project files are Installed:

1. Create a [Github](#) directory if you are familiar with Git
2. Ensure all project files are on your current system (in the predetermined directory structure), and inside your local Git pathway (if you decided to use Git)

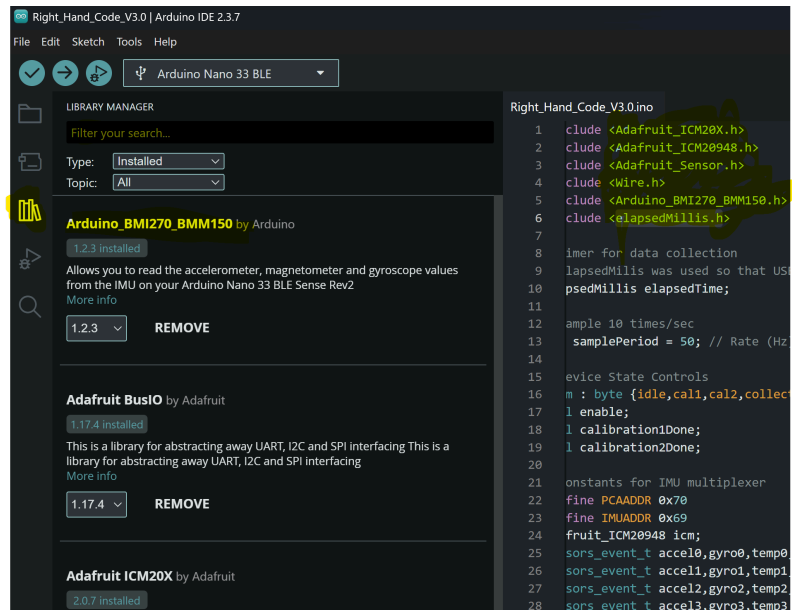
Now, we will begin setting up our program. First, we will make the necessary installations inside of our Arduino IDE.

Current Tasks ~ Install Packages and Board:

1. Open Arduino IDE. You can either do this by opening Right_Hand_Code_V3.0.ino from file explorer (Should be in '<Your_Project_Directory>\Technical Addendum\Code\Arduino\Right_Hand_Code_V3.0'), or by running the Arduino application directly.
2. We will begin by installing our board (the arduino controller used in the gloves) into our arduino IDE. The board is called 'Arduino Nano 33 BLE'. Install it by going into the 'boards manager' menu and searching for it directly:



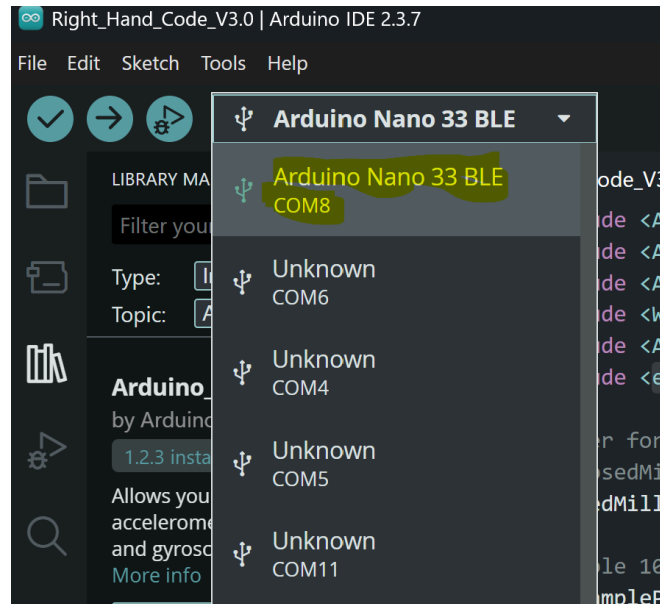
3. Next, we will want to install the libraries within our file `Right_Hand_Code_V3.0.ino`. I would recommend opening it inside of Arduino IDE right now.
 4. The necessary packages are as follows: - `Adafruit_ICM20X.h` - `Adafruit_Sensor.h` - `Arduino_BMI270_BMM150.h` - `elapsedMillis.h` - `ICM20948_WE` - `Adafruit BusIO`
- Typing these one at a time into the library manager will enable you to download them all. If it ever prompts you along the lines of: ‘do you wish to download dependencies,’ say ‘**YES.**’



Now, we are ready to figure out which COM our glove connects to. NOTE THAT YOU SHOULD NOT HAVE TO PROGRAM THE GLOVE AT THIS TIME. Simply plug the glove in, and figure out which port arduino sees your board on. Alternatively, you can figure this out by using device manager. Both methods are demonstrated below:

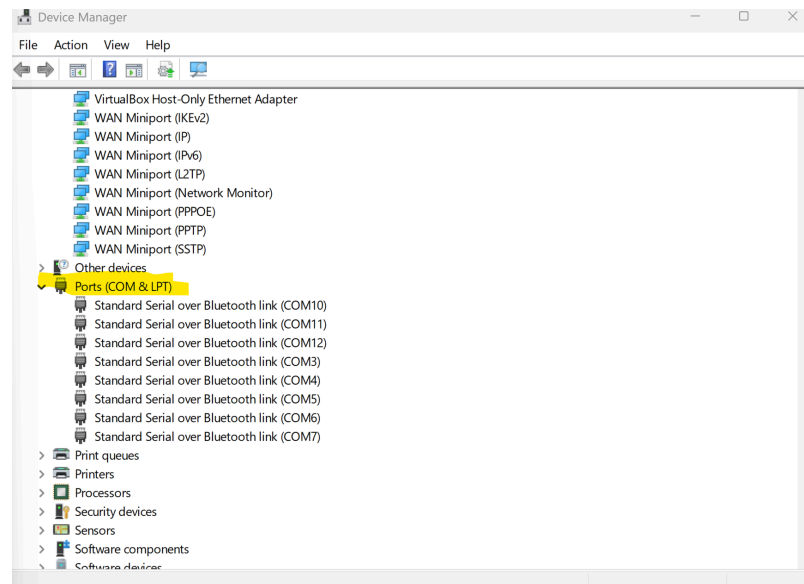
Current Tasks ~ Install Packages and Board:

1. Note that in my case, the glove is on 'COM8'. Yours will likely be different. Additionally, the different hands (right and left) will likely also have different COM numbers even if you plug them into the same physical USB port on your computer at different times. But in these examples, the correct answer is 'COM8'.
2. Using Arduino IDE: Simply use the dropdown as follows and verify the detected device:

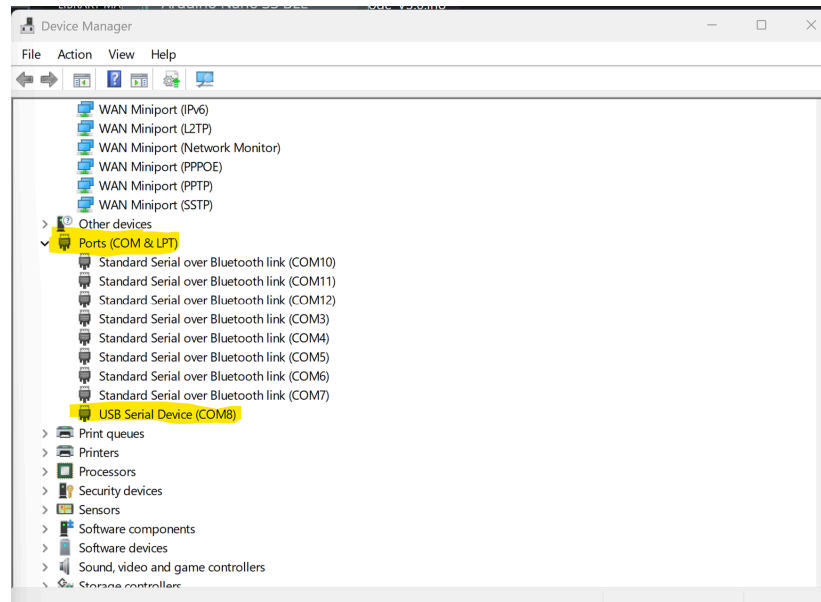


Note that you may have to scroll

3. Using Device Manager: Unplug your device and open device manager. Open the expandable menu called Ports (COM & LPT):



4. Using Device Manager: Now, plug your glove back in and observe which port is created (will likely be called 'USB Serial Device (COM#)' as seen below):



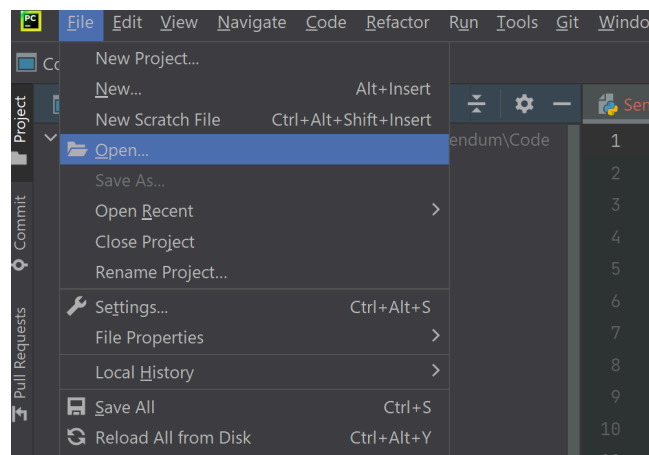
5. Write this port down somewhere (maybe notepad)

Now, it is time for us to begin working with Pycharm. First, please ensure you have [Python](#) installed on your system. Take note of your exact installation pathway. The easiest way to find this is by opening powershell and typing: (Get-Command python).Path

I will not elaborate further on the installation of Python. Please open Pycharm. Within Pycharm, you will want to open our project directory. To do this, click 'File, Open':

Current Tasks ~ Open Pycharm, Open Project:

1. Open Pycharm
2. Open Your Project (I recommend opening in a new window):



3. Within our project directory, open ‘\Technical Addendum\Code\Python’.
This is where our relevant code is kept.

Once you have the project opened in Pycharm, you will need to install all runtime-related packages. The list of required packages is below:

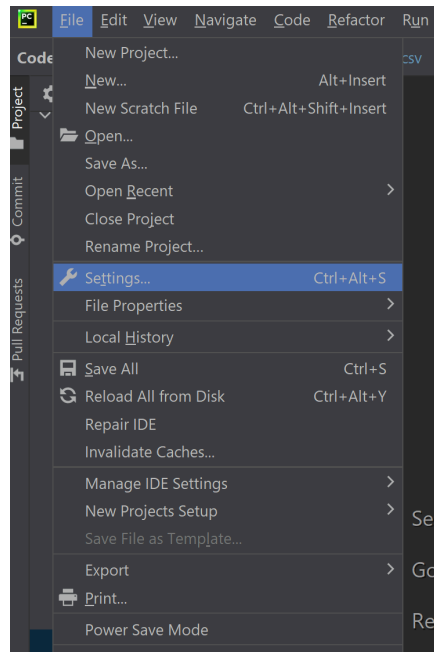
- serial
- csv
- time
- tkinter
- threading
- sys
- math
- collections
- scipy
- PySide6
- numpy

If you already have [Anaconda Navigator](#) installed and know how to create and use a Python environment, I recommend doing so for this project. Otherwise, I will quickly demonstrate how to install all packages on your local Python interpreter. Please remember the file path to your Python installation.

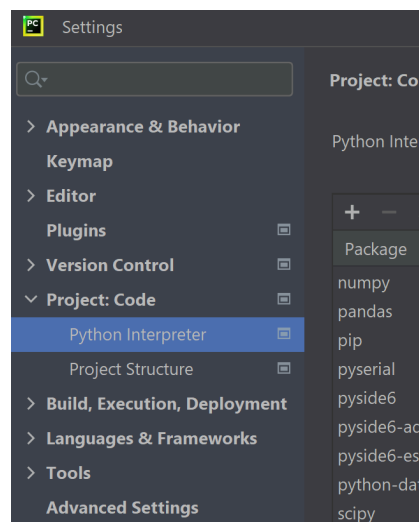
We will now assign a Python interpreter to our current project instance. To do so, follow the steps below:

Current Tasks ~ Create Complete Python Interpreter:

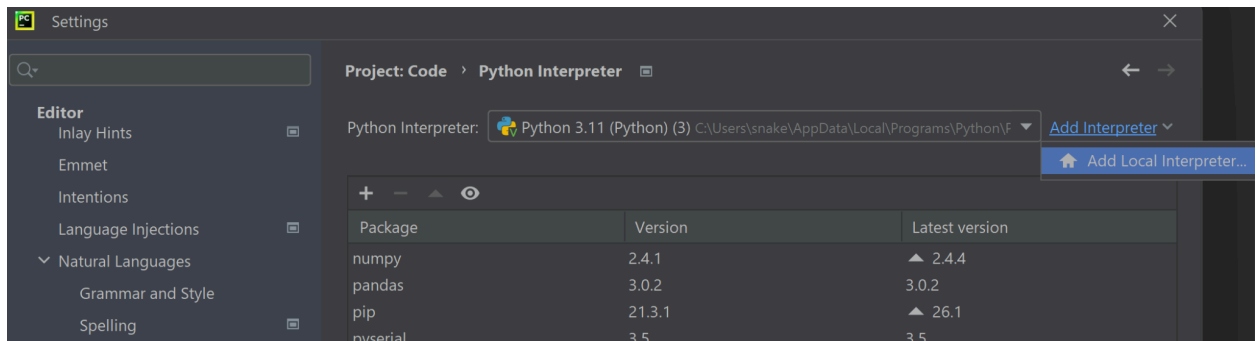
1. Open your Pycharm settings in 'File, Settings':



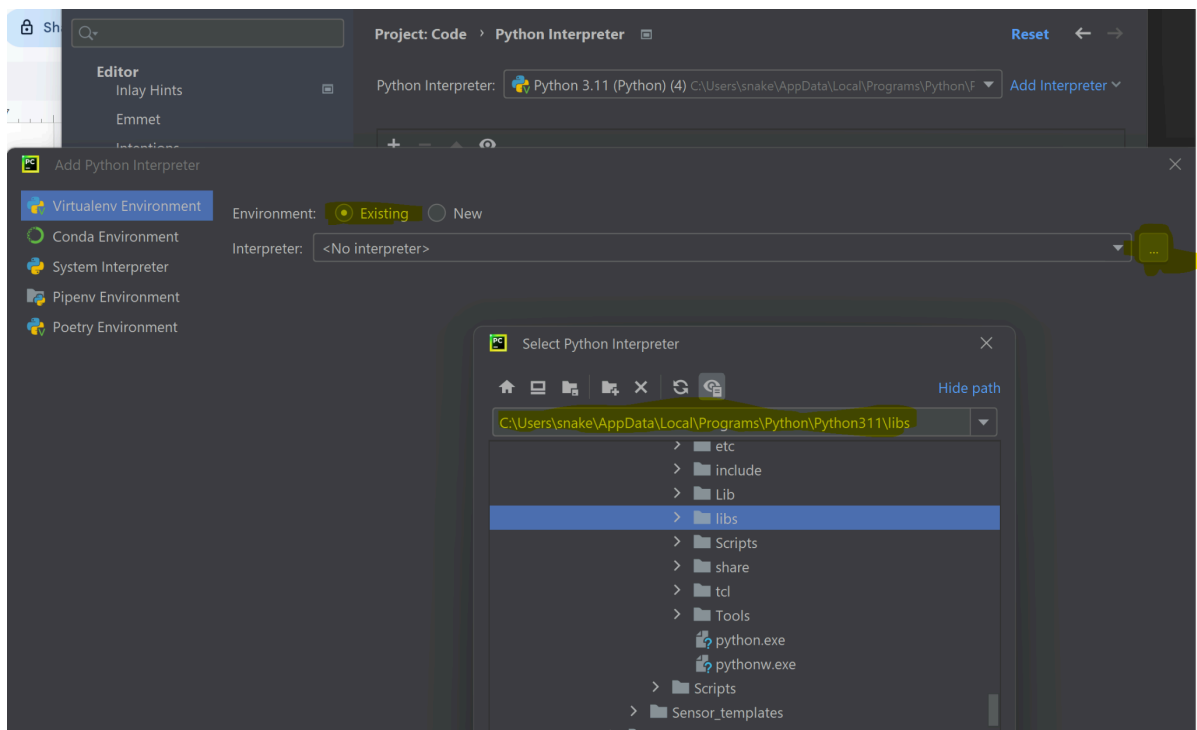
2. Find the setting for configuring Python Interpreters in 'Project: Code, Python Interpreter':



3. Click 'Add Interpreter, Add Local Interpreter':



4. If you would like to create a new Interpreter, that is fine as long as you note the directory in which it is created for future use (DO NOT RELY ON PYCHARM TO TRACK THIS). Otherwise, add an existing interpreter (either the base python installation or the environment you created using Anaconda Navigator):



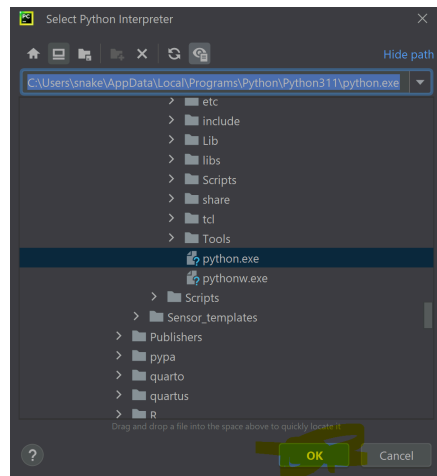
5. Paste your exact pathway into the section highlighted yellow, and press enter. Again (if not using a dedicated environment), this may be found using Powershell:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

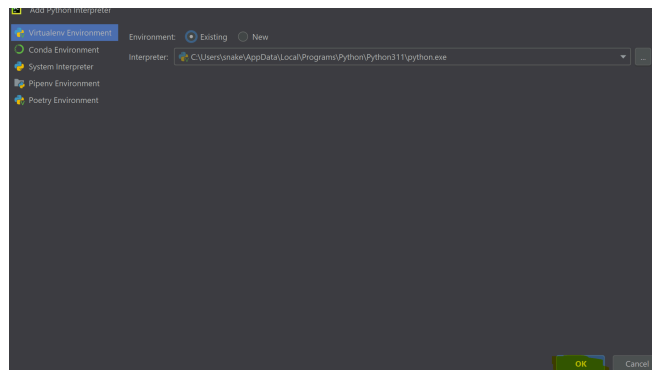
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\sna...> (Get-Command python).Path
C:\Users\sna...AppData\Local\Programs\Python\Python311\python.exe
PS C:\Users\sna...> |
```

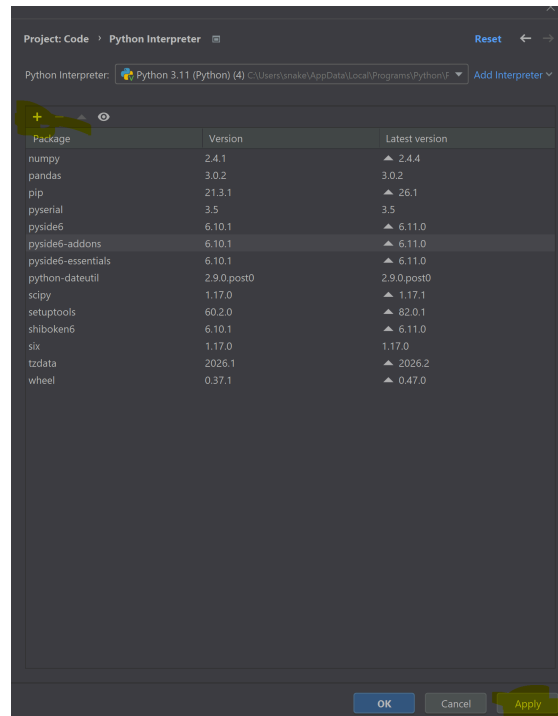
6. Select 'OK':



7. Select 'OK':

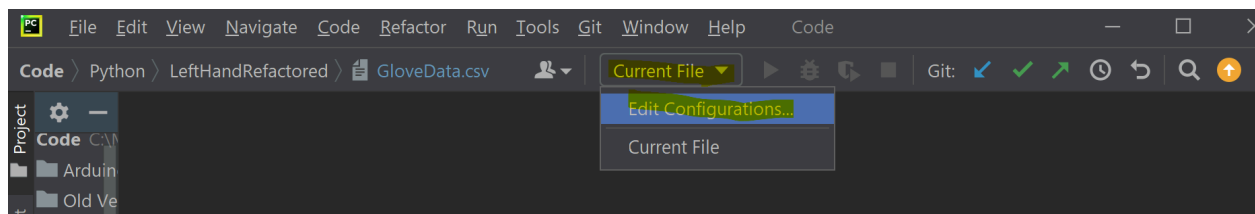


8. Select 'Apply', Then you may use the '+' to add all packages from the list above (Page 6).

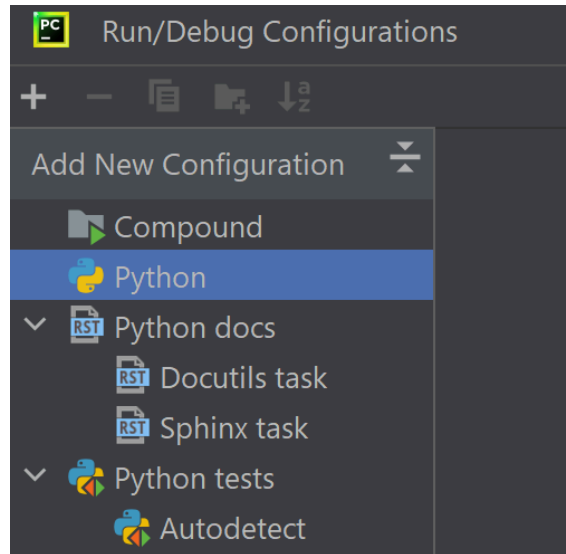


9. Once you have added all packages, Click ‘Apply’ once more, then click ‘OK’

10. Now we will apply our environment. Select ‘Current File, Edit Configurations’:



11. Click the ‘+’, select ‘Python’:

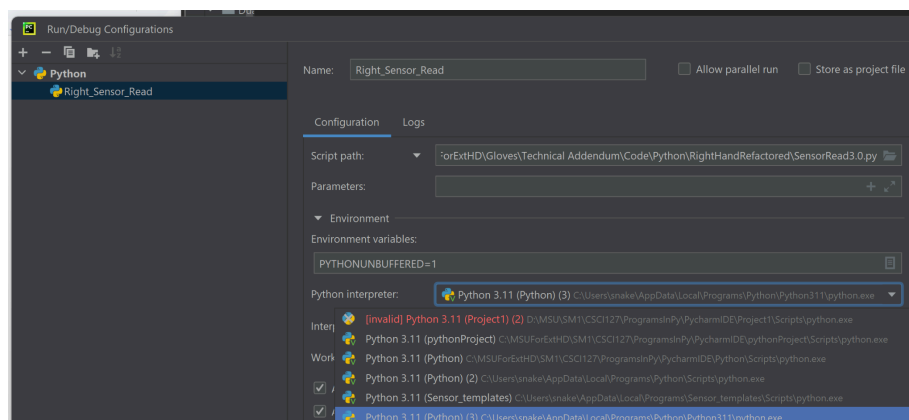


12. Name it 'Right_Sensor_Read,' Give it script path

'<path_to_project>\Technical

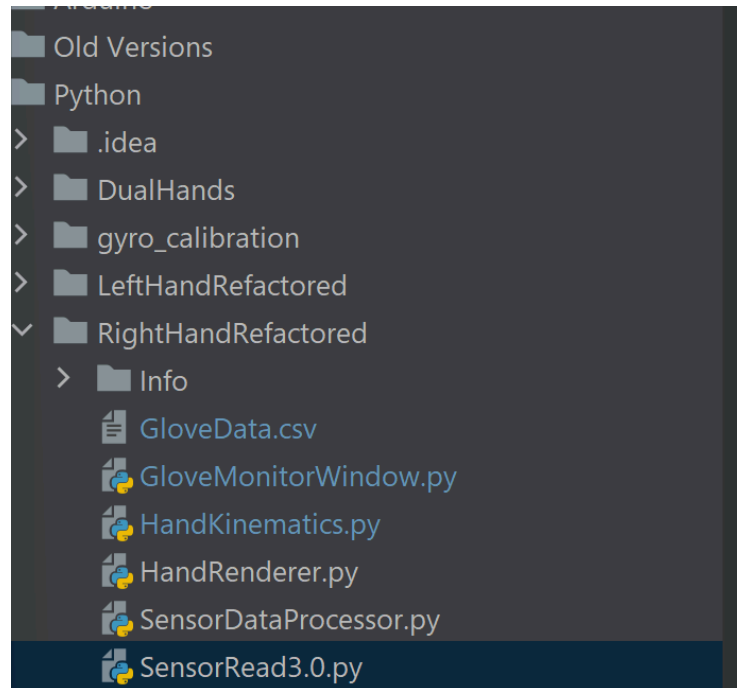
Addendum\Code\Python\RightHandRefactored\[SensorRead3.0.py](#),'

Select your configured Python Interpreter:



13. Click 'Apply', then 'OK'.

14. Plug the Right-handed glove into your computer. Additionally, I recommend updating the code to your correct COM Port. Inside of your Pycharm Application, Open 'RightHandRefactored\[SensorRead3.0.py](#)':

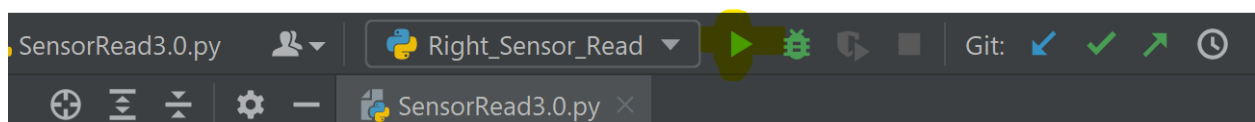


15. Edit the code directly to match your port:

```
import serial
import csv
import time
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox
import threading
import sys
from GloveMonitorWindow import GloveMonitorWindow

#Serial port constants and variables
port = 'COM8'
baudRate = 2000000
dataLine = [0]
dataThread = None
reader = None
```

16. Click Run (green triangle next to insect):



Operation

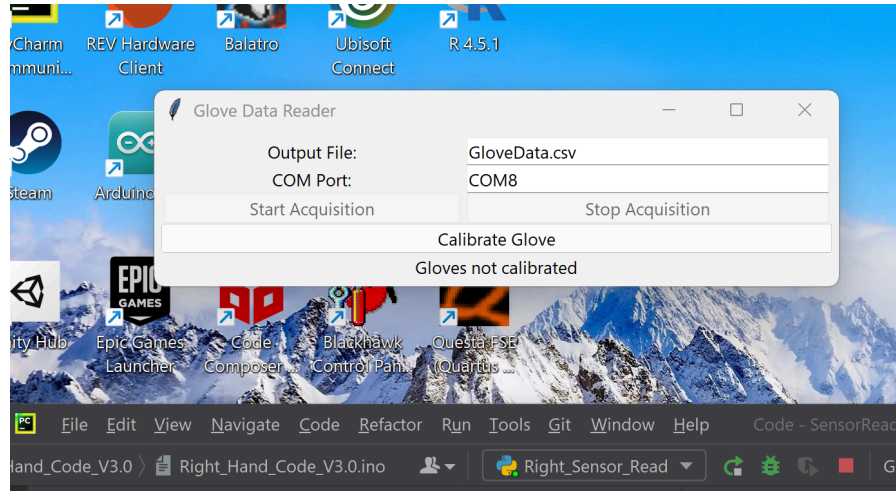
Now, you have the code running. This is very exciting. In the current implementation, most runtime operations are relatively intuitive and there is not much risk of you breaking anything. Still, I recommend reviewing the following notes to gain the proper and exact context. One useful tip is to simply unplug the glove and then force-stop the program if behavior becomes weird. This has never failed to properly stop the program, and has not caused any issues for the arduino. Additionally, the glove should be unplugged any time the program restarts.

The first thing you will see is the tkinter window, opened from [SensorRead3.0.py](#), and is largely unchanged from the original senior design project which kickstarted this process. Before being allowed to sample, you must begin by calibrating the glove's flex sensors. To do this, follow the prompt instructions to an extreme (borderline painful) degree.

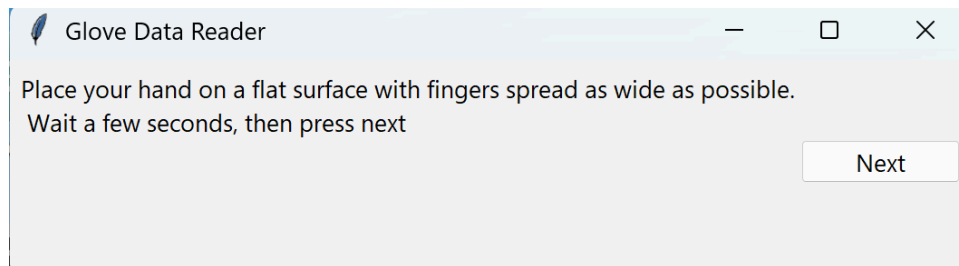
As a developer note, you may want to consider looking at '\Technical Addendum\Code\Arduino\Right_Hand_Code.V3.0' and using some form of averaging (I recommend selecting the extreme from a group of samples, actually) in your calibration process. The arduino code had other problems to fix when I started with it, and I decided this was not a priority because I was able to get it working most of the time. My general rule for editing the arduino code was to leave the left hand unchanged until I verified total good operation in the right hand, then making corresponding changes to the left. **THE ONLY DIFFERENCE BETWEEN THE RIGHT HANDED AND LEFT HANDED CODE SHOULD BE THE CSV DATA COLUMN DESCRIBING WHICH IS WHICH. THUS FAR IN DEVELOPMENT, I HAVE USED THIS DETAIL FOR NO PURPOSE WHATSOEVER.**

Current Tasks ~ Calibrate Gloves:

1. Find the Tkinter window. This may take a moment to appear, and may appear behind your Pycharm Window:

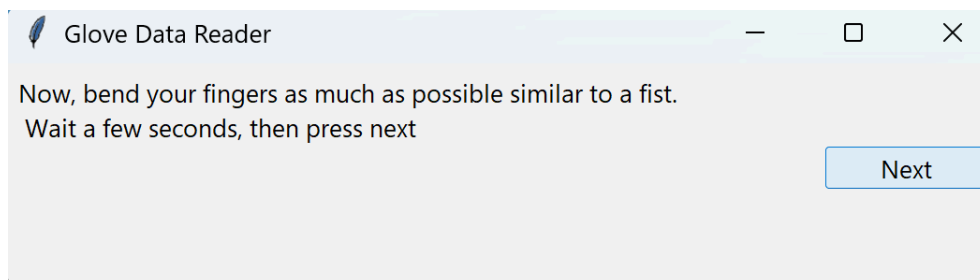


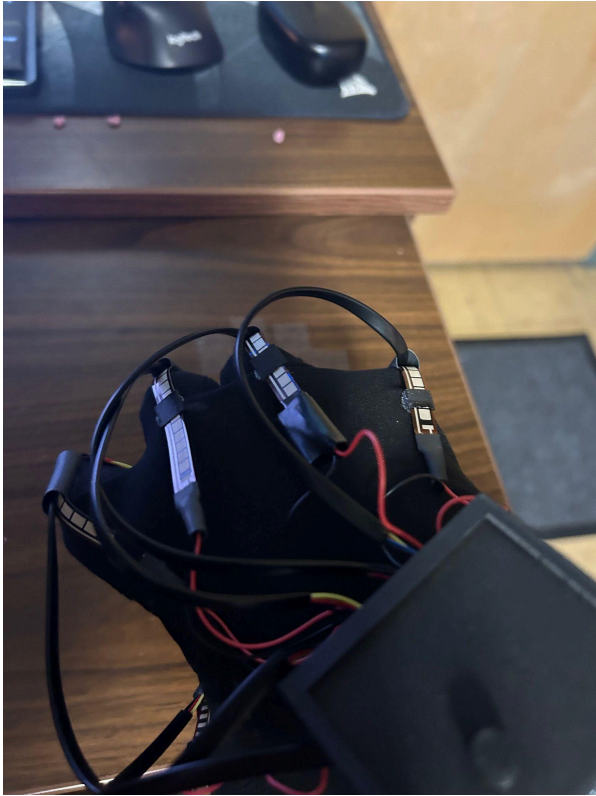
2. Click 'Calibrate Glove' (before doing this, ensure you have your glove plugged into the computer, and that the COM# text listed in COM Port is correct)
3. Follow the step below. Lifting your fingers is advisable.:





4. Follow the step below. You should exaggerate the flex sensors to the greatest degree to which you are capable:



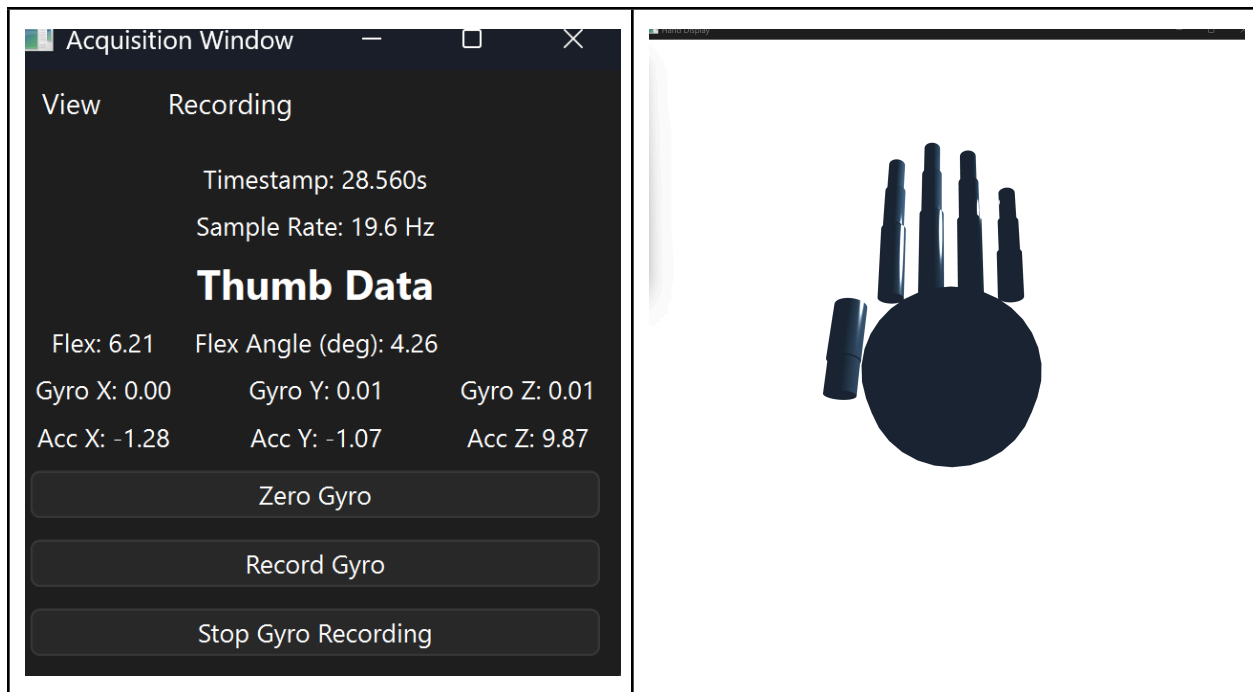


5. Click Finish

Now, we are ready to begin our recording. The following will be a general description of the current user options and menu.

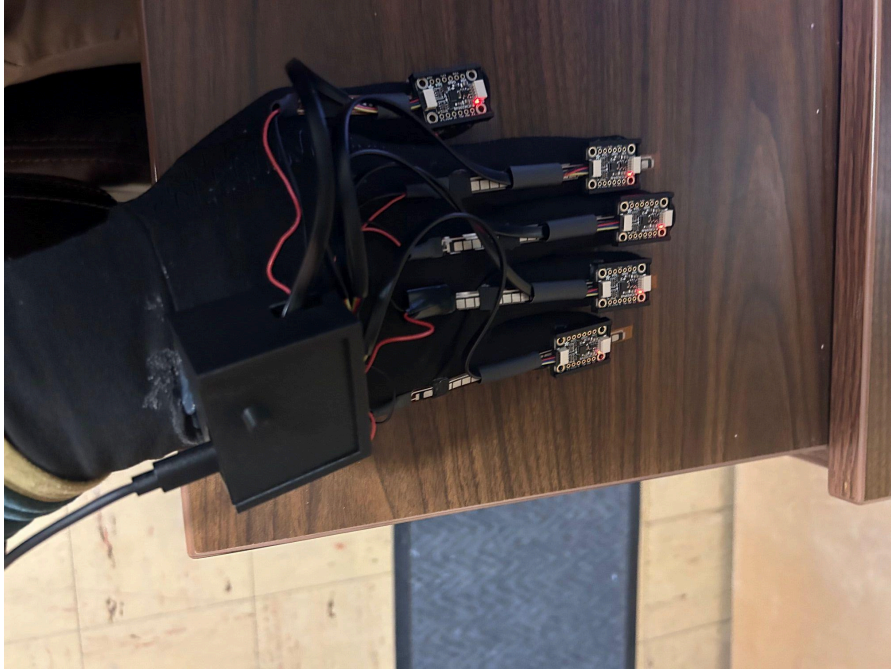
Guide ~ Using the Interface ~ Startup:

1. Click 'Start Acquisition'. A Pyside general window and a corresponding animation window displaying the glove should appear (sometimes, the top light source fails to initialize. This seems completely random and I have no idea why it happens):



2. If the fingers appear bent backwards, start by clicking 'zero gyro.' If this does not help, restart the program by either unplugging the gloves and manually stopping the program, or by clicking 'stop acquisition' and clicking the 'x' in the top of the tkinter window (this will get old).

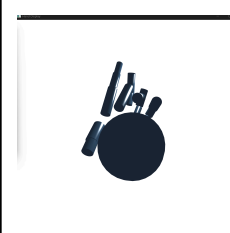
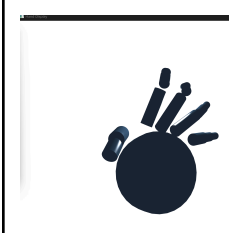
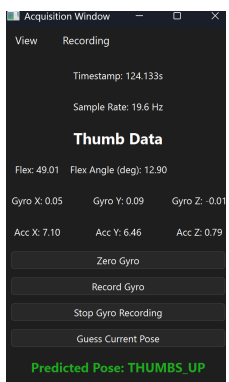
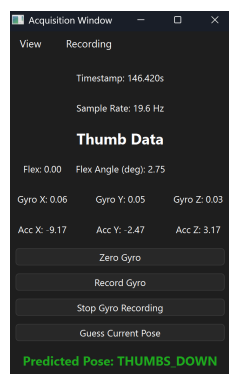
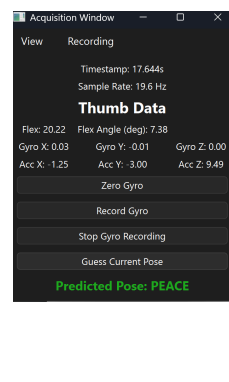
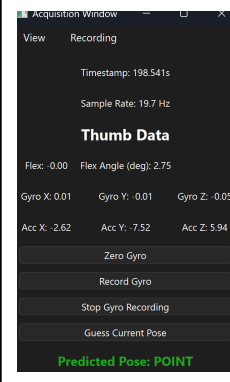
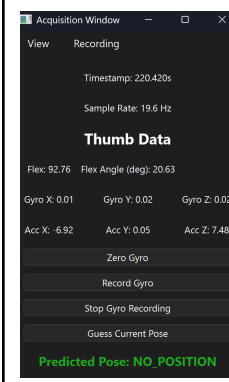
3. Once you have appropriately calibrated your flex sensors, put your hand on a flat surface and click 'zero gyro' in the pyside general window.



4. If your computer is lagging, you can reduce the demand by adjusting the `ANIMATION_SAMPLE_RATE_MULTIPLIER` variable inside of [GloveMonitorWindow.py](#). This determines the ratio between glove samples and animation frames.

Guide ~ Using the Interface:

1. The most entertaining aspect of the interface I have added is the 'Recording → Identify' section. If my model still approximately works for you, you should be able to identify 'thumbs up', 'thumbs down', 'peace', 'point' and 'no position' with relatively decent accuracy. I was able to achieve very good results here with just ~210 total samples. Improving the Gyroscope self-correction matrices and the overall glove design should improve this further.

Thumbs Up	Thumbs Down	Peace	Point	No Position
				
				

- The other aspects of the interface are primarily for utility. They include general data display (raw flex values useful for manual linear regression sampling to determine values for `finger_flex_to_angle` and `thumb_flex_to_angle` in [SensorDataProcessor.py](#)) which you can change from the ‘view’ dropdown, continuous position recording to CSV format (currently pointless) from the ‘recording → Continuous’ selection (output file `PositionData_Continuous.csv`), and labeled snapshot position recording in ‘recording → Snapshots’ with output file `PositionData_Snapshots.csv`. I will elaborate slightly on these below:
- The raw display was used to verify signal stability and general behavior. Additionally, it was used in the general decision making process for determining how to decipher the signal. Finally, it can still be used to manually evaluate real-time performance metrics (sampling rate should be stable around 19.6 Hz, uptime should track cleanly, and flex data behavior should successfully be converted from polynomial to linear behavior). You may change which component of the hand you are seeing by clicking ‘View → X’, where ‘X’ may be ‘Thumb’, ‘Pointer’, ‘Middle’, ‘Ring’, ‘Pinky’, or ‘Wrist’. The default display data is of the thumb.

4. Continuous position recording is essentially what our animation window is used to validate. Getting this to be somewhat reliable took up the bulk of my worktime. I am not doing anything with this data currently, but it could likely be modified to post-run position identification, or gesture identification (see ‘suggested next steps’) later on.
5. Taking snapshots is relatively easy. Simply move your hand into the appropriate position and click the corresponding button. Addition of additional positions should be relatively trivial. To train a new identifier model, move your ‘PositionData_Snapshots.csv’ file to the /Position_Identification/ folder and run `rain_hand_pose_model.py`. This should not take long, but you will need corresponding packages:
 - pandas
 - sklearn
 - joblib
 - os

After your model has been trained (`hand_pose_random_forest_model.pkl`), simply move it to the /RightHandRefactored/models/ folder.

Developer Tools

I have hinted towards the existence of several useful tools I outlined during my process. These include the gyroscope self-correcting matrices, flex sensor linear regression, and position identification training.

Developer Guide ~ Flex Sensor Linear Regression:

File Directory: \Gloves\Flex-Sensor-Angles\

Files: ‘flex sensor angles.xlsx’, ‘pointer.m’, ‘thumb.m’

This is relatively simple. We simply record flex-sensor angles directly (go 0 through 170 degree bend angles for fingers, 0 through 140 for thumb, 5 degree increments) and read the corresponding flex sensor values from your zeroed positions. We store these combinations *somewhere* and run our values through `pointer.m` and `thumb.m`. I suggest doing this with the gloves on.

File Directory: \Gloves\Technical Addendum\Code\Python\RightHandRefactored\
Following this, find 'finger_to_flex_angle(flex_value)' and
'finger_to_flex_angle(flex_value)' functions in [SensorDataProcessor.py](#) inside of the and
update the corresponding return values to whatever the Matlab output specifies for
coefficients (3rd order polynomial).

I personally just used a basic angle measuring tool for this. It is a clear, circular object. I
left it in the kit for you.

Developer Guide ~ Gyroscope Self-Correction:

The existing method for fixing the gyroscope error is made up of two general
components for sampling: raw zero-movement drift and return-to-zero drift.
Existing files with samples can be found in '\Gloves\Technical Addendum\Code\Old
Versions\Python\Erik_Depracted\' . Feel free to use these for testing. Still, do not neglect
the following instructions:

File Directory: \Gloves\Technical Addendum\Code\Python\RightHandRefactored\
To record a session of gyroscope angular speed values, simply run the code, locate the
main Pyside window, and press 'Record Gyro.' To end the recording, click 'Stop Gyro
Recording'. This creates 6 files named 'gyro_log2.csv' (wrist), 'gyro_log_thumb.csv',
'gyro_log_point.csv', 'gyro_log_mid.csv', 'gyro_log_ring.csv' and 'gyro_log_pink.csv'.
For more information, see the FILE_MAP variable inside of the GyroRecorder class
within the SensorDataProcessor.py file.

To actually create useful recordings, create some type of marker on a flat surface. This
should be sufficient for you to return your hand to the exact physical position repeatedly
without fail, while wearing the glove. For testing, I put tape around the palm area and
where every finger tip needed to sit. For true refinement, I suggest using some type of
mold (even dried play-dough would suffice) which allows even more reliability with less
focus.

Once you have this repeatable reference point, you can begin the first step of sampling: place your hand on the reference point. You may zero the gyroscope for personal testing, but this is not actually necessary because all we record here are the raw sensor values adjusted to radians and filtered through our lowpass filter.

Now, you simply click 'Record Gyro.' Move the hand around a little bit, then move it back to the original position. click 'Stop Gyro Recording'. This is one sample. Hundreds to thousands of samples may be required for sufficiently correct gyroscope error. Finally, take samples over differing durations from around 2 up to 60 seconds (no limit, but this is likely near the limit of usability). Ensure these samples incorporate a variety of motions on every axis, at differing speeds, and in differing rotation directions.

The next step is to get raw sensor drift without movement. again in [SensorDataProcessor.py](#), find the FILE_MAP variable inside of the GyroRecorder class (line ~189. Notice that below it, there is a commented definition of this variable with '_s' appended to each name. This '_s' denotes 'stationary' to us. Uncomment this replacement file-naming scheme.

Re-run the program, but this time, hold your hand perfectly still from the beginning until the end of recordings. We will calculate the raw sensors' drifts as the averages of these variables. This recording does not need to be as extensive, and ~200 seconds of total recording time should be sufficient.

File Directory: \Gloves\Technical Addendum\Code\Python\gyro_calibration\

Within this directory, you will find \data\raw\ containing files gyro_log.csv and gyro_stationary.csv. These are the only two csv file names which should ever appear here. Starting with the wrist (this had the greatest success in my experience), copy the content of your gyro_log_X.csv file into gyro_log.csv. Then, copy the content of your corresponding gyro_log_X_s.csv file into gyro_stationary.

Finally, the code may require some adjustment depending upon your file. Because the current design mounted the wrist gyroscopes upside-down, our proportional target matrices need to account for this negative speed. So, in \calibration\, you need to make 2 edits: in files [loss.py](#) and [train.py](#), a matrix called A_REF exists. For fingers, you simply select the identity 3x3 matrix. However, for the wrist, you must utilize:

1	0	0
0	1	0
0	0	-1

as A_REF. This line already exists in both files (at the top) and just needs to be uncommented (while commenting the identity one).

There are substantial system requirements for this file, and it may take a very long time to run. I recommend utilizing linear models for our correction and temporarily ignoring the quadratic option. In fact, you may comment out the quadratic calls from main to save time.

Specifically, your system needs at least 8 usable CPU threads (for 8 folds) and a solid amount of memory (depends on the number of samples you have). If you do not have a desktop computer with at least a 13th generation Intel CPU (or equivalent), I recommend running this on Tempest (The school's super computer). It may be accessed remotely. For further elaboration, please contact your advisor.

The packages required are as follows:

- numpy
- scipy
- os
- multiprocessing
- sklearn
- pandas

Download these, then run the program from [main.py](#).

The significant outputs will be 3 data matrices (along with approximate error correction), or 2 if (and we will be using) you use the linear model. Manually save these values to `\data\raw\recent_transformations_matrices.txt`.

Finally, you may implement these matrices into your code. Right now, I exclusively transform the incoming wrist data. To see how, go to `/RightHandRefactored/HandKinematics.py`. Within the `RightHand` class, inside the `__init__()` method, find variables `self._A_opt` and `self._b_opt`. Replace these values with your new wrist matrices. To see their application, find `update_orientation()` and look at the `gyro_corrected` variable. For fingers, create your own corresponding implementation inside of `update_orientation_fingers()` (I did not yet, but this is relatively easy considering the method's input structure).

Run the regular program from `SensorRead3.0` and verify nothing broke. See if you notice any improvement in drift or the frequency at which you must zero the sensors.

- **DO NOT MAKE ANY ALTERATIONS TO PRECEDING OR FOLLOWING SECTIONS OF CODE WITHOUT A SUFFICIENT UNDERSTANDING OF QUATERNIONS**

Repeat this process for all fingers. If a component stops moving, ensure values on your `A` transformation matrix diagonals are not zeroed or nearly-zeroed. This is technically a trivial solution to our problem, as our program is meant to optimize in drift reduction over intervals and no movement → no drift. If you run into this issue, go back to `/gyro_calibration/main.py` and find `LAMBDA_A`, `LAMBDA_B` variables and increase them. The penalize deviation from our target matrices (we do not want substantial change from baseline behavior, just optimized error correction). These parameters are near the top of our file.

Developer Guide ~ Position Identification:

This has already been partially explained, so I will keep it brief.

Required Packages:

- pandas
- sklearn
- joblib
- os

Steps:

1. run SensorRead3.0.py, go to your main Pyside window, click 'Recording → Snapshots'
2. Look at your epic choice of positions. Make that position with your hand and click the corresponding button. This has created one sample.
3. Take at least 100 samples for basic model success. More for usable accuracy. Then, stop the program.
4. Move PositionData_Snapshots to /Position_Identification/. Run train_hand_pose_model.readme. Move created hand_pose_random_forest_model.pkl to the /models/ folder
5. restart your main program, running from [SensorRead3.0.py](#). Open your Pyside Menu
6. Click 'Recording → identify'
7. Make a position with your hand and click 'Guess Current Pose'. It should display an estimate of your current position

Suggested Next Steps

1. Total Glove Physical Redesign:

For all intents and purposes, this needs to happen before further development should continue. The current platform is not worth optimizing the configuration upon. Additionally, it would be best to advance the prototype before we lose our working reference (the existing gloves

will wear out). Therefore, this should be done as soon as possible (assuming this project continues).

The following issues must be resolved:

- Load-bearing wires ~ Presently, wires connecting to flex sensors operate and have no reinforcement whatsoever. I have needed to re-solder them manually, and these issues will only exacerbate themselves with extended usage. Additionally, wires connecting to gyroscope/accelerometer boards on fingertips become unplugged relatively easily, which is not ideal for extended usage.
- Glued components ~ The glove's most fundamental structure is held together with hot glue and super glue. 3-D parts should contain thread holes for direct stitching or some other mechanical attachment
- Gloves versatility ~ The current gloves are not flexible enough in size, or easy enough to put on. The fabric has and will continue to stretch and wear over time.
- Wired connection to computer ~ not super pressing, but utilizing a wireless communication module should be practical and would improve ease of use long-term.
- Floating boards and sensors ~ Currently, not enough is done to secure the arduino, gyroscope/acceleration sensors, and flex sensors. Wires are not long enough to enable true freedom when moving hands around. Additionally, components are not secure enough in general (gyro and flex error partially caused by physical instability).
- Modular design ~ at present, the main board and sensor attachments exist in a halfway-accessible mounting state. Ideally, components should be shelved individually, improving both access and stability.
- Real PCBs ~ Actual manufactured PCBs should be utilized for wiring purposes, no matter how simplistic. Proof-of-concept and verification now exists, so a more robust and cleaner platform investment is worthwhile.
- Exposed components ~ These gloves are intended for use partially for children; particularly, children who struggle comprehending speech and therefore instructions. They should appear relatively uninteresting with limited exposed

parts to pick at or mess around with. The present design is not only destructible, it actually encourages ‘exploration.’

Ideas for Redesign:

- All components should be pocketed some way in the fabric. The wires should be long enough to leave plenty of leeway while the sensors themselves remain completely static on the gloves.
- The only part that should remain wholly fabric is the top-side of the glove. The underside should mostly consist of velcro, buckled, or some other form of adjustable attachments and fasteners. This way, taking them on/off just requires setting the main glove on-top of the hand while attaching the fasteners.
- Electrical wires could be reinforced with slightly shorter generic wire that follows the same path
- Stacked PCBs/expansion boards in the main chassis (3d printed case) could be extended to either travel up the wrist or forward on the hand, thus shortening the compartment
- Flex sensors could be put into a firm position (right now they get pulled backward/pushed forward), with a mechanical adjustment mechanism (primarily for sizing) if necessary.
- Gloves should not cause any fatigue or stress on the hand. Ensure you have redundant fasteners to prevent any form of concentrated stress.
- For now, ignore the weight. The highest priority is still testing, so anything that improves the structural integrity, general versatility, and/or sensor stability/consistency is a boon.
- Make sure you don’t mount sensors upside down. When I received them, this inconsistency in mounting was a major oversight.

2. More Extensive Gyroscope Self-Zeroing:

- You will want to (at a minimum) complete linear correction matrices for all gyroscope finger and wrist sensors. If this seems insubstantial, other options do exist, such as:
 - The accelerometers sense gravity as well. Develop an AI model to determine when movement ceases, then use gravity detection to re-orient stored quaternions

such that gravity is down. NOTE: current design has wrist gyroscope and accelerometer mounted upside down.

- Some other form of exact testing for sensor error. My current design requires user sampling with sensors still attached to gloves, but you could theoretically improve this correction precision by mounting to a stepper motor and recording exact rotation over time with a chemical/mechanical level to perfectly angle the motor on axis.

3. More Extensive Position Identification:

- Verify that position can be extended to almost all feasible distinctions. Try all sign language letters, for example. Maybe test precision by seeing if the model can determine which direction is being pointed in (already somewhat verified with thumbs up/down, with sideways being identified as ‘no position’).
- Depending on the issues or limitations you encounter, it may be advisable to revert to previous design stages before moving forward with gesture identification.

4. Gesture Identification:

- This is considered one of the main points of the project. I will lay out my ideas below:
 - Gestures should be tracked as combinations of identifiable positions. The model should identify positions in real-time, while a second model takes those positions and learns to associate them with defined gestures (i.e. wave could be identified as ‘flat hand vertical-left’ + ‘flat hand vertical’ + ‘flat hand vertical-right’ + ‘flat hand vertical’)
 - Training might involve using post-recording position identification combined with real-time frames of ‘this gesture X happened within this timestamp.’ with the inputs being all identified positions within that timestamp and the target being the actual gesture performed.
- If this succeeds, it might be good to develop a model to search for patterns in positions, enabling it to define its own positions over individual datasets. This could improve versatility into positions that we do not think of on our own, or that are simply abstract repeated positions. This could also pick up on an individual’s ticks and quirks, telling us more about an individual person compared to a universal gesture system.

5. Gesture Live-Tracking:

- Gesture identification should begin on prerecorded data. Relying on model throughput speed to find them in real time would be a mistake, initially. However, it may become useful later if we can get reliable live results. Optimization ideas and practicality analysis are left to you.

6. Gesture Post-Operation Tracking:

- This should be easier than #5, but it will become critical that sessions can be properly analyzed post-operation. This is likely the gloves' final cornerstone before true live deployment attempts.

etc. Quality of Life:

- Update your arduino files to take multiple samples while calibrating. Maybe choose more accurate default fall-backs as well.
- Feel free to use LLMs such as Claude or Grok to work with code. You must know exactly what you want changed before prompting, however. I have used AI at several stages to suggest packages, implement trivial changes, implement complex changes (with heavy description and validation), and to re-factor code. Take an iterative approach to this, knowing your logical sequence of events without providing LLMs irrelevant information. They will save you time if used properly, or they may rapidly turn this project into AI slop if used improperly. They may also be useful for understanding the code.. well, just don't use OpenAI because ChatGPT loves to give hyper-abstract descriptions and pass them off as something approximating comprehension (it never is).